

```

# import the necessary modules and libraries
import json, unittest, datetime

#use the open function to open read the three json files
with open("./data-1.json","r") as f:
    jsonData1 = json.load(f)
with open("./data-2.json","r") as f:
    jsonData2 = json.load(f)
with open("./data-result.json","r") as f:
    jsonExpectedResult = json.load(f)

# convert json data from format 1 to the expected format
def convertFromFormat1 (jsonObject):

    # IMPLEMENT: Conversion From Type 1
    locationParts = jsonObject["location"].split("/")
    #create a new dictionary for the unified format
    result= {
        'deviceID': jsonObject['deviceID'],
        'deviceType': jsonObject['deviceType'],
        'timestamp': jsonObject['timestamp'],
        'location': {
            'country': locationParts[0], #extract the country from the location string
            'city': locationParts[1], #extract the city from the location string
            'area': locationParts[2], #extract the area from the location string
            'factory': locationParts[3], #extract the factory from the location string
            'section': locationParts[4] #extract the section from the location string
        },
        'data': {
            'status': jsonObject['operationStatus'], #copy the operationStatus to status
            'temperature': jsonObject['temp'] #copy the temp to temperature
        }
    }

    return result

# convert json data from format 2 to the expected format
def convertFromFormat2 (jsonObject):

    # IMPLEMENT: Conversion From Type 1
    # convert the ISO 8601 timestamp to milliseconds since epoch
    data= datetime.datetime.strptime(jsonObject['timestamp'], '%Y-%m-%dT%H:%M:%S.%fZ') #ISO 8601
    format
    timestamp = round((data - datetime.datetime(1970, 1, 1)).total_seconds() * 1000) #convert to
    milliseconds since epoch

    #create a new dictionary for the unified format
    result={
        'deviceID': jsonObject['device']['id'], #extract the device ID
        'deviceType': jsonObject['device']['type'], #extract the device type
        'timestamp': timestamp, #use the converted timestamp
        'location': {
            'country': jsonObject['country'], #copy the country
            'city': jsonObject['city'], #copy the city
            'area': jsonObject['area'], #copy the area
            'factory': jsonObject['factory'], #copy the factory
            'section': jsonObject['section'] #copy the section
        },
        'data': jsonObject['data'] #copy the entire data object
    }

    return result

```

```
def main (jsonObject):

    result = {}

    if (jsonObject.get('device') == None):
        result = convertFromFormat1(jsonObject)
    else:
        result = convertFromFormat2(jsonObject)

    return result


# Test cases using unittest module
class TestSolution(unittest.TestCase):

    # Sanity test to ensure the expected result is as intended
    # converts json data to python objects usnig json.loads and json.dumps
    def test_sanity(self):

        result = json.loads(json.dumps(jsonExpectedResult))
        self.assertEqual(
            result,
            jsonExpectedResult
        )

    def test_dataType1(self):

        result = main (jsonData1)
        self.assertEqual(
            result,
            jsonExpectedResult,
            'Converting from Type 1 failed'
        )

    def test_dataType2(self):

        result = main (jsonData2)
        self.assertEqual(
            result,
            jsonExpectedResult,
            'Converting from Type 2 failed'
        )

if __name__ == '__main__':
    # run the tests
    unittest.main()
```